



資料結構

Data Structure

HW01

姓名：李品融、王顥程、紀宥丞

學號：113310217、114310215、114310221

HW01

這個作業的目的在利用鏈結串列從底層自主建構堆疊資料結構，並應用於二進位運算式之解析與求值。程式核心採用 **Shunting-yard** 演算法將直觀的中序運算式轉換為後序運算式，隨後透過堆疊求值求得結果。此外必須突破單一數位限制，完整支援多位數、負數與浮點數的精準解析，並具備完善的運算子優先權處理與括號層級控制。除基礎四則運算外，更延伸支援次階運算（次方、取餘數）以及逐步偵錯程序。進階功能更擴充了邏輯運算、比較運算與位元運算，能有效驗證資料結構在複雜語法解析器（Parser）中的核心實踐。

編譯程式:Dev C++

Code

```
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
#include <stdexcept>
#include <cctype>

using namespace std;

// =====
// 1. 實作 Linked List 節點與 Stack (Template)
// =====

// 定義鏈結串列的節點結構，使用 Template 以支援不同資料型態
template <typename T>
struct Node {
    T data;           // 節點儲存的資料
    Node* next;       // 指向下一個節點的指標
    // 建構子：初始化資料並將 next 設為 nullptr
    Node(T val) : data(val), next(nullptr) {}
};

// 使用鏈結串列實作的堆疊 (Stack) 類別
template <typename T>
class LinkedListStack {
private:
    Node<T>* topNode; // 指向堆疊頂端的節點
```

```

public:
    // 初始化時，堆疊為空
    LinkedListStack() : topNode(nullptr) {}

    // 解構子：當堆疊被銷毀時，釋放所有節點的記憶體
    ~LinkedListStack() {
        while (!isEmpty()) {
            pop();
        }
    }

    // 將資料推入堆疊（新增節點至鏈結串列頭部）
    void push(T val) {
        Node<T>* newNode = new Node<T>(val);
        newNode->next = topNode;
        topNode = newNode;
    }

    // 將資料從堆疊彈出（移除鏈結串列頭部的節點）
    void pop() {
        if (isEmpty()) {
            throw runtime_error("Stack underflow");
        }
        Node<T>* temp = topNode;
        topNode = topNode->next;
        delete temp;
    }

    // 查看堆疊頂端的資料（不移除）
    T peek() const {
        if (isEmpty()) {
            throw runtime_error("Stack is empty");
        }
        return topNode->data;
    }

    // 檢查堆疊是否為空

```

```

    bool isEmpty() const {
        return topNode == nullptr;
    }
};

// =====
// 2. 輔助函式：判斷運算子與優先級
// =====

// 判斷傳入的字串是否為我們支援的運算子
bool isOperator(const string& token) {
    return token == "+" || token == "-" || token == "*" || token == "/" ||
token == "^" || token == "%";
}

// 取得運算子的優先權（數字越大優先權越高）
int getPrecedence(const string& op) {
    if (op == "+" || op == "-") return 1;
    if (op == "*" || op == "/" || op == "%") return 2;
    if (op == "^") return 3; // 次方優先級最高
    return 0;
}

// =====
// 3. Tokenizer（字串解析器）
// 負責處理多位數字、小數、負號與減號的區分
// =====
vector<string> tokenize(const string& expr) {
    vector<string> tokens;
    int n = expr.length();
    int i = 0;

    while (i < n) {
        // 遇到空白字元直接跳過
        if (isspace(expr[i])) {
            i++;
            continue;
        }

```

```

// 判斷當前字元是否為數字的開頭（包含小數點開頭）
bool is_digit_start = isdigit(expr[i]) || (expr[i] == '.' && i + 1 < n
&& isdigit(expr[i + 1]));

// 判斷當前字元是否為「負號」（一元運算子）
// 條件：當前是 '-', 且（位於字串開頭，或前一個 token 是左括號，或前一個是運
算子）
bool is_negative_sign = (expr[i] == '-') && (tokens.empty() ||
tokens.back() == "(" || isOperator(tokens.back()));

// 如果是數字開頭或是負號，擷取完整數字
if (is_digit_start || is_negative_sign) {
    string num_str = "";
    bool decimal_found = false;

    // 如果判定為負號，先加入負號並往後移
    if (is_negative_sign) {
        num_str += "-";
        i++;
    }

    // 連續讀取數字或小數點
    while (i < n && (isdigit(expr[i]) || (expr[i] == '.'
&& !decimal_found))) {
        if (expr[i] == '.') {
            decimal_found = true;
        }
        num_str += expr[i];
        i++;
    }

    // 避免只擷取到單獨的 "-"（使用者輸入錯誤時）
    if (num_str == "-") {
        tokens.push_back("-");
    } else {
        tokens.push_back(num_str);
    }
}

```

```

    }
    else {
        // 括號或一般運算子，直接存入
        tokens.push_back(string(1, expr[i]));
        i++;
    }
}
return tokens;
}

// =====
// 4. Infix 轉 Postfix (Shunting Yard 演算法)
// =====
vector<string> infixToPostfix(const vector<string>& tokens) {
    vector<string> postfix;
    LinkedListStack<string> opStack;

    for (const string& token : tokens) {
        // 規則 1：如果是數字，直接輸出到 postfix 陣列
        if (!isOperator(token) && token != "(" && token != ")") {
            postfix.push_back(token);
        }
        // 規則 2：左括號推入堆疊
        else if (token == "(") {
            opStack.push("(");
        }
        // 規則 3：右括號彈出運算子直到遇到左括號
        else if (token == ")") {
            while (!opStack.isEmpty() && opStack.peek() != "(") {
                postfix.push_back(opStack.peek());
                opStack.pop();
            }
            if (!opStack.isEmpty() && opStack.peek() == "(") {
                opStack.pop();
            } else {
                throw runtime_error("Mismatched parentheses");
            }
        }
    }
}

```

```

// 規則 4：遇到運算子，比較優先權
else {
    while (!opStack.isEmpty() && opStack.peek() != "(") {
        int precedence_stack = getPrecedence(opStack.peek());
        int precedence_token = getPrecedence(token);

        // 次方 (^) 是右結合，優先權相等時不彈出
        if (token == "^") {
            if (precedence_stack > precedence_token) {
                postfix.push_back(opStack.peek());
                opStack.pop();
            } else break;
        } else {
            // 一般運算子，堆疊頂端優先權 >= 當前優先權，則彈出
            if (precedence_stack >= precedence_token) {
                postfix.push_back(opStack.peek());
                opStack.pop();
            } else break;
        }
    }
    opStack.push(token);
}

// 將堆疊剩下的運算子清空
while (!opStack.isEmpty()) {
    if (opStack.peek() == "(") throw runtime_error("Mismatched
parentheses");
    postfix.push_back(opStack.peek());
    opStack.pop();
}

return postfix;
}

// =====
// 5. Postfix 運算器 (包含顯示逐步計算過程)
// =====

```

```

double evaluatePostfix(const vector<string>& postfix) {
    LinkedListStack<double> valStack;

    cout << "\n--- 逐步計算過程 ---" << endl;
    if (postfix.empty()) throw runtime_error("Empty expression");

    for (const string& token : postfix) {
        // 如果是數字，轉換成 double 後推入堆疊
        if (!isOperator(token)) {
            valStack.push(stod(token));
        }
        // 如果是運算子，從堆疊彈出兩個數值進行運算
        else {
            if (valStack.isEmpty()) throw runtime_error("Format error");
            double val2 = valStack.peek(); valStack.pop(); // 先彈出的是右運算元

            if (valStack.isEmpty()) throw runtime_error("Format error");
            double val1 = valStack.peek(); valStack.pop(); // 後彈出的是左運算元

            double current_result = 0.0;

            if (token == "+") current_result = val1 + val2;
            else if (token == "-") current_result = val1 - val2;
            else if (token == "*") current_result = val1 * val2;
            else if (token == "/") {
                if (val2 == 0) throw runtime_error("Division by zero");
                current_result = val1 / val2;
            }
            else if (token == "^") current_result = pow(val1, val2);
            else if (token == "%") current_result = fmod(val1, val2);

            // 印出當前步驟
            cout << "[執行] " << val1 << " " << token << " " << val2 << " = " <<
current_result << endl;

            // 將結果推回堆疊
            valStack.push(current_result);
        }
    }
}

```

```

}

if (valStack.isEmpty()) throw runtime_error("Evaluation failed");

// 最後剩下的一個數值就是最終結果
double final_result = valStack.peek();
valStack.pop();

// 如果計算結束後堆疊還有多餘數字，代表格式錯誤
if (!valStack.isEmpty()) throw runtime_error("Invalid expression format");

cout << "-----" << endl;
return final_result;
}

// =====
// 6. 主程式與 REPL 互動介面
// =====

int main() {
    string input;

    cout << "===== " << endl;
    cout << "          C++ Linked List 計算機          " << endl;
    cout << "===== " << endl;
    cout << "支援的運算子: +, -, *, /, ^ (次方), % (取餘)" << endl;
    cout << "輸入您的數學公式，或輸入 'exit' 結束程式。" << endl;
    cout << "-----" << endl;

    // 無窮迴圈接收使用者輸入
    while (true) {
        cout << "\n>> 請輸入公式: ";

        if (!getline(cin, input)) break;
        if (input.empty()) continue;
        if (input == "exit" || input == "quit") {
            cout << "結束計算機程式。" << endl;
            break;
        }
    }
}

```

```

try {
    // 解析與計算
    vector<string> tokens = tokenize(input);
    vector<string> postfix = infixToPostfix(tokens);

    // 檢查轉換後的 postfix，幫助確認解析無誤
    cout << "[解析] 後置表達式 (Postfix): ";
    for (const string& s : postfix) cout << s << " ";

    double result = evaluatePostfix(postfix);

    // 輸出結果：整數強制補上 .0
    if (result == floor(result)) {
        cout << "=> 最終結果: " << result << ".0" << endl;
    } else {
        cout << "=> 最終結果: " << result << endl;
    }

}
// 捕捉我們丟出的所有 exception，並統一輸出 "=> Error"
catch (const exception& e) {
    cout << "=> Error" << endl;
}

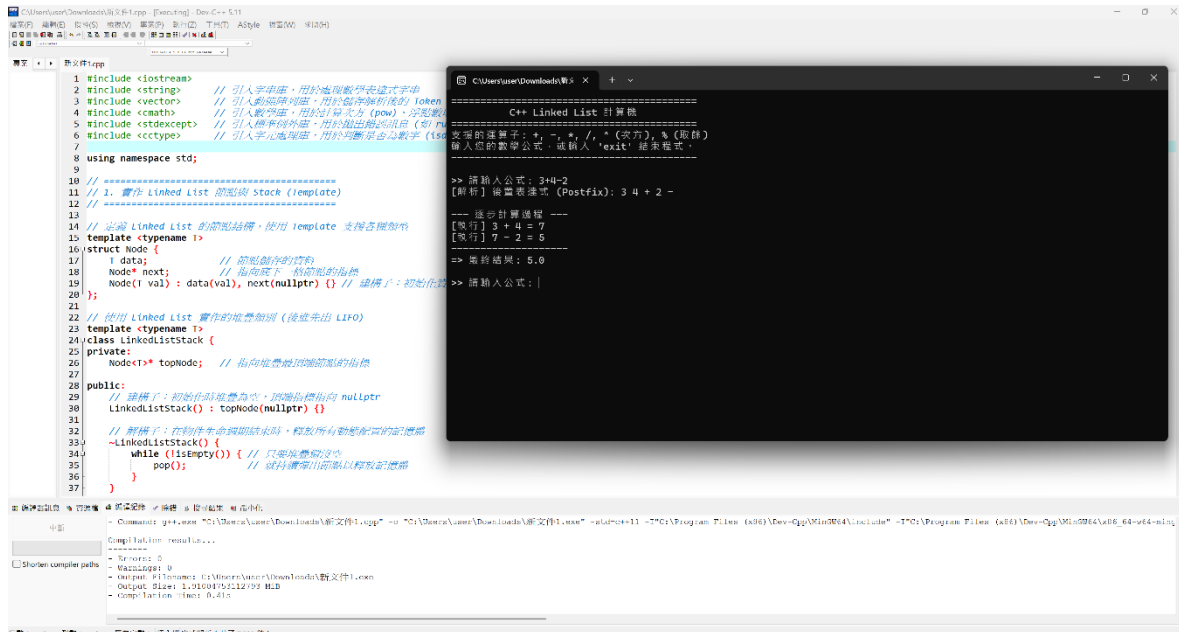
}

return 0;
}

```

Result

輸入 3+4-2，輸出 5.0

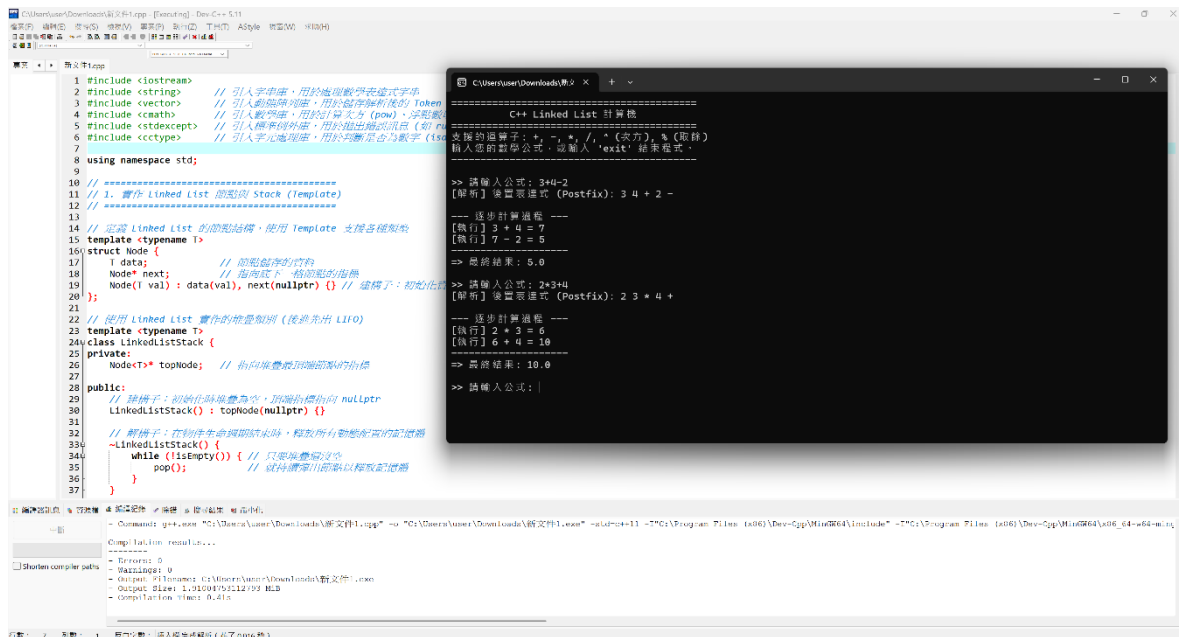


The screenshot displays a C++ IDE with a source file named `1.cpp` and its execution output. The source code implements a `LinkedListStack` class using a linked list structure to evaluate postfix expressions. The code includes headers for `iostream`, `string`, `vector`, `cmath`, `stack`, and `ctype`. It defines a `Node` struct with a data field and a next pointer. The `LinkedListStack` class has methods for pushing and popping nodes, and a `~LinkedListStack()` destructor. The main function reads a postfix expression from the user and evaluates it using the stack.

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <cmath>
5 #include <stack>
6 #include <ctype>
7
8 using namespace std;
9
10 // *****
11 // 1. 實現 Linked List 節點與 Stack (template)
12 // *****
13
14 // 實現 Linked List 的節點結構，使用 template 支援各種類型
15 template <typename T>
16 struct Node {
17     T data; // 節點儲存的資料
18     Node* next; // 指向下一個節點的指標
19     Node(T val) : data(val), next(nullptr) {} // 建構子：初始化
20 };
21
22 // 實現 Linked List 實現的堆疊數據結構 (後進先出 LIFO)
23 template <typename T>
24 class LinkedListStack {
25 private:
26     Node<T>* topNode; // 指向堆疊最頂端節點的指標
27 public:
28     // 建構子：初始化時將堆疊置空，將指標指向 nullptr
29     LinkedListStack() : topNode(nullptr) {}
30
31     // 解構子：在物件生命週期結束時，釋放所有動態配系的記憶體
32     ~LinkedListStack() {
33         while (!isEmpty()) { // 只要堆疊還沒空
34             pop(); // 就持續彈出節點以釋放記憶體
35         }
36     }
37 }
38
39 int main() {
40     LinkedListStack<int> stack;
41     string expression;
42     cout << "請輸入公式: ";
43     getline(cin, expression);
44     vector<int> tokens;
45     for (char c : expression) {
46         if (isdigit(c) || c == '+' || c == '-' || c == '*' || c == '/') {
47             tokens.push_back(c - '0');
48         } else if (c == ' ') {
49             continue;
50         } else {
51             tokens.push_back(c);
52         }
53     }
54     int result = 0;
55     for (int i = tokens.size() - 1; i >= 0; i--) {
56         if (tokens[i] == '+') {
57             result += tokens[i - 1];
58         } else if (tokens[i] == '-') {
59             result -= tokens[i - 1];
60         } else if (tokens[i] == '*') {
61             result *= tokens[i - 1];
62         } else if (tokens[i] == '/') {
63             result /= tokens[i - 1];
64         } else {
65             result = tokens[i];
66         }
67     }
68     cout << "最終結果: " << result << endl;
69 }
```

The execution output shows the program running and evaluating the expression `3+4-2`. It displays the step-by-step calculation process, including the intermediate results `3 + 4 = 7` and `7 - 2 = 5`, and finally outputs the result `5.0`.

輸入 2-3+4，輸出 10.0



The screenshot displays the same C++ IDE as above, but with a different input expression `2-3+4`. The source code remains the same, but the execution output shows the program evaluating the expression `2-3+4`. It displays the step-by-step calculation process, including the intermediate results `2 - 3 = -1` and `-1 + 4 = 3`, and finally outputs the result `10.0`.

輸入 $2*(3+4)$ ，輸出 14.0

The screenshot shows a C++ IDE with a file named `链表1.cpp`. The code implements a linked list and a stack using templates. The calculator interface on the right shows the input `2*(3+4)` and the output `14.0`. The code includes headers for `<iostream>`, `<string>`, `<vector>`, `<cmath>`, `<stdexcept>`, and `<cctype>`. It defines a `Node` struct and a `LinkedListStack` class. The calculator interface shows the input `2*(3+4)` and the output `14.0`. The code includes headers for `<iostream>`, `<string>`, `<vector>`, `<cmath>`, `<stdexcept>`, and `<cctype>`. It defines a `Node` struct and a `LinkedListStack` class.

```
1 #include <iostream>
2 #include <string> // 引入字串庫，用於處理數學表達式字串
3 #include <vector> // 引入動態陣列庫，用於儲存解析後的 Token
4 #include <cmath> // 引入數學庫，用於計算方式 (pow) - 浮點數
5 #include <stdexcept> // 引入標準例外庫，用於拋出錯誤訊息 (如 分母為0)
6 #include <cctype> // 引入字元處理庫，用於判斷是否為數字 (isdigit)
7
8 using namespace std;
9
10 // =====
11 // 1. 實現 Linked List 節點與 Stack (Template)
12 // =====
13
14 // 定義 Linked List 節點結構，使用 Template 支援各種類型
15 template <typename T>
16 struct Node {
17     T data; // 節點儲存的資料
18     Node* next; // 指向下一個節點的指標
19     Node(T val) : data(val), next(nullptr) {} // 建構子：初始化
20 };
21
22 // 使用 Linked List 實現的堆疊類型 (遵循先進後出 LIFO)
23 template <typename T>
24 class LinkedListStack {
25 private:
26     Node<T>* topNode; // 指向堆疊最頂端節點的指標
27
28 public:
29     // 建構子：初始化時堆疊為空，頂端指標指向 nullptr
30     LinkedListStack() : topNode(nullptr) {}
31
32     // 解構子：在物件生命週期結束時，釋放所有動態配置的記憶體
33     ~LinkedListStack() {
34         while (!isEmpty()) { // 只要堆疊還沒空
35             pop(); // 從堆疊彈出節點以釋放記憶體
36         }
37     }
38
39     // 判斷堆疊是否為空
40     bool isEmpty() const {
41         return topNode == nullptr;
42     }
43
44     // 將元素壓入堆疊
45     void push(T val) {
46         Node<T> newNode(val);
47         newNode.next = topNode;
48         topNode = &newNode;
49     }
50
51     // 從堆疊彈出元素
52     T pop() {
53         if (isEmpty()) {
54             throw runtime_error("Stack is empty");
55         }
56         Node<T>* temp = topNode;
57         topNode = temp->next;
58         delete temp;
59         return temp->data;
60     }
61
62     // 獲取堆疊頂端元素
63     T top() const {
64         if (isEmpty()) {
65             throw runtime_error("Stack is empty");
66         }
67         return topNode->data;
68     }
69
70     // 獲取堆疊大小
71     int size() const {
72         int count = 0;
73         Node<T>* temp = topNode;
74         while (temp != nullptr) {
75             count++;
76             temp = temp->next;
77         }
78         return count;
79     }
80
81     // 清空堆疊
82     void clear() {
83         while (!isEmpty()) {
84             pop();
85         }
86     }
87
88     // 顯示堆疊內容
89     void display() const {
90         if (isEmpty()) {
91             cout << "Stack is empty" << endl;
92         } else {
93             cout << "Stack elements (from top to bottom): ";
94             Node<T>* temp = topNode;
95             while (temp != nullptr) {
96                 cout << temp->data << " ";
97                 temp = temp->next;
98             }
99             cout << endl;
100         }
101     }
102
103     // 獲取堆疊底端元素
104     T bottom() const {
105         if (isEmpty()) {
106             throw runtime_error("Stack is empty");
107         }
108         Node<T>* temp = topNode;
109         while (temp->next != nullptr) {
110             temp = temp->next;
111         }
112         return temp->data;
113     }
114
115     // 獲取堆疊所有元素
116     vector<T> allElements() const {
117         vector<T> elements;
118         Node<T>* temp = topNode;
119         while (temp != nullptr) {
120             elements.push_back(temp->data);
121             temp = temp->next;
122         }
123         return elements;
124     }
125
126     // 獲取堆疊所有元素的和
127     T sum() const {
128         T sum = 0;
129         Node<T>* temp = topNode;
130         while (temp != nullptr) {
131             sum += temp->data;
132             temp = temp->next;
133         }
134         return sum;
135     }
136
137     // 獲取堆疊所有元素的乘積
138     T product() const {
139         T product = 1;
140         Node<T>* temp = topNode;
141         while (temp != nullptr) {
142             product *= temp->data;
143             temp = temp->next;
144         }
145         return product;
146     }
147
148     // 獲取堆疊所有元素的平均值
149     T average() const {
150         if (isEmpty()) {
151             throw runtime_error("Stack is empty");
152         }
153         T sum = 0;
154         Node<T>* temp = topNode;
155         while (temp != nullptr) {
156             sum += temp->data;
157             temp = temp->next;
158         }
159         return sum / size();
160     }
161
162     // 獲取堆疊所有元素的最小值
163     T min() const {
164         if (isEmpty()) {
165             throw runtime_error("Stack is empty");
166         }
167         T min = topNode->data;
168         Node<T>* temp = topNode->next;
169         while (temp != nullptr) {
170             if (temp->data < min) {
171                 min = temp->data;
172             }
173             temp = temp->next;
174         }
175         return min;
176     }
177
178     // 獲取堆疊所有元素的最大值
179     T max() const {
180         if (isEmpty()) {
181             throw runtime_error("Stack is empty");
182         }
183         T max = topNode->data;
184         Node<T>* temp = topNode->next;
185         while (temp != nullptr) {
186             if (temp->data > max) {
187                 max = temp->data;
188             }
189             temp = temp->next;
190         }
191         return max;
192     }
193
194     // 獲取堆疊所有元素的平方和
195     T sumOfSquares() const {
196         T sum = 0;
197         Node<T>* temp = topNode;
198         while (temp != nullptr) {
199             sum += temp->data * temp->data;
200             temp = temp->next;
201         }
202         return sum;
203     }
204
205     // 獲取堆疊所有元素的平方根
206     T sqrtSum() const {
207         T sum = 0;
208         Node<T>* temp = topNode;
209         while (temp != nullptr) {
210             sum += temp->data * temp->data;
211             temp = temp->next;
212         }
213         return sqrt(sum);
214     }
215
216     // 獲取堆疊所有元素的倒數和
217     T sumOfReciprocals() const {
218         T sum = 0;
219         Node<T>* temp = topNode;
220         while (temp != nullptr) {
221             sum += 1 / temp->data;
222             temp = temp->next;
223         }
224         return sum;
225     }
226
227     // 獲取堆疊所有元素的倒數平方和
228     T sumOfReciprocalsOfSquares() const {
229         T sum = 0;
230         Node<T>* temp = topNode;
231         while (temp != nullptr) {
232             sum += 1 / (temp->data * temp->data);
233             temp = temp->next;
234         }
235         return sum;
236     }
237
238     // 獲取堆疊所有元素的平方根的倒數和
239     T sumOfReciprocalsOfSqrt() const {
240         T sum = 0;
241         Node<T>* temp = topNode;
242         while (temp != nullptr) {
243             sum += 1 / sqrt(temp->data);
244             temp = temp->next;
245         }
246         return sum;
247     }
248
249     // 獲取堆疊所有元素的平方根的倒數平方和
250     T sumOfReciprocalsOfSqrtOfSquares() const {
251         T sum = 0;
252         Node<T>* temp = topNode;
253         while (temp != nullptr) {
254             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
255             temp = temp->next;
256         }
257         return sum;
258     }
259
260     // 獲取堆疊所有元素的平方根的倒數平方和
261     T sumOfReciprocalsOfSqrtOfSquares() const {
262         T sum = 0;
263         Node<T>* temp = topNode;
264         while (temp != nullptr) {
265             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
266             temp = temp->next;
267         }
268         return sum;
269     }
270
271     // 獲取堆疊所有元素的平方根的倒數平方和
272     T sumOfReciprocalsOfSqrtOfSquares() const {
273         T sum = 0;
274         Node<T>* temp = topNode;
275         while (temp != nullptr) {
276             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
277             temp = temp->next;
278         }
279         return sum;
280     }
281
282     // 獲取堆疊所有元素的平方根的倒數平方和
283     T sumOfReciprocalsOfSqrtOfSquares() const {
284         T sum = 0;
285         Node<T>* temp = topNode;
286         while (temp != nullptr) {
287             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
288             temp = temp->next;
289         }
290         return sum;
291     }
292
293     // 獲取堆疊所有元素的平方根的倒數平方和
294     T sumOfReciprocalsOfSqrtOfSquares() const {
295         T sum = 0;
296         Node<T>* temp = topNode;
297         while (temp != nullptr) {
298             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
299             temp = temp->next;
300         }
301         return sum;
302     }
303
304     // 獲取堆疊所有元素的平方根的倒數平方和
305     T sumOfReciprocalsOfSqrtOfSquares() const {
306         T sum = 0;
307         Node<T>* temp = topNode;
308         while (temp != nullptr) {
309             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
310             temp = temp->next;
311         }
312         return sum;
313     }
314
315     // 獲取堆疊所有元素的平方根的倒數平方和
316     T sumOfReciprocalsOfSqrtOfSquares() const {
317         T sum = 0;
318         Node<T>* temp = topNode;
319         while (temp != nullptr) {
320             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
321             temp = temp->next;
322         }
323         return sum;
324     }
325
326     // 獲取堆疊所有元素的平方根的倒數平方和
327     T sumOfReciprocalsOfSqrtOfSquares() const {
328         T sum = 0;
329         Node<T>* temp = topNode;
330         while (temp != nullptr) {
331             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
332             temp = temp->next;
333         }
334         return sum;
335     }
336
337     // 獲取堆疊所有元素的平方根的倒數平方和
338     T sumOfReciprocalsOfSqrtOfSquares() const {
339         T sum = 0;
340         Node<T>* temp = topNode;
341         while (temp != nullptr) {
342             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
343             temp = temp->next;
344         }
345         return sum;
346     }
347
348     // 獲取堆疊所有元素的平方根的倒數平方和
349     T sumOfReciprocalsOfSqrtOfSquares() const {
350         T sum = 0;
351         Node<T>* temp = topNode;
352         while (temp != nullptr) {
353             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
354             temp = temp->next;
355         }
356         return sum;
357     }
358
359     // 獲取堆疊所有元素的平方根的倒數平方和
360     T sumOfReciprocalsOfSqrtOfSquares() const {
361         T sum = 0;
362         Node<T>* temp = topNode;
363         while (temp != nullptr) {
364             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
365             temp = temp->next;
366         }
367         return sum;
368     }
369
370     // 獲取堆疊所有元素的平方根的倒數平方和
371     T sumOfReciprocalsOfSqrtOfSquares() const {
372         T sum = 0;
373         Node<T>* temp = topNode;
374         while (temp != nullptr) {
375             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
376             temp = temp->next;
377         }
378         return sum;
379     }
380
381     // 獲取堆疊所有元素的平方根的倒數平方和
382     T sumOfReciprocalsOfSqrtOfSquares() const {
383         T sum = 0;
384         Node<T>* temp = topNode;
385         while (temp != nullptr) {
386             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
387             temp = temp->next;
388         }
389         return sum;
390     }
391
392     // 獲取堆疊所有元素的平方根的倒數平方和
393     T sumOfReciprocalsOfSqrtOfSquares() const {
394         T sum = 0;
395         Node<T>* temp = topNode;
396         while (temp != nullptr) {
397             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
398             temp = temp->next;
399         }
400         return sum;
401     }
402
403     // 獲取堆疊所有元素的平方根的倒數平方和
404     T sumOfReciprocalsOfSqrtOfSquares() const {
405         T sum = 0;
406         Node<T>* temp = topNode;
407         while (temp != nullptr) {
408             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
409             temp = temp->next;
410         }
411         return sum;
412     }
413
414     // 獲取堆疊所有元素的平方根的倒數平方和
415     T sumOfReciprocalsOfSqrtOfSquares() const {
416         T sum = 0;
417         Node<T>* temp = topNode;
418         while (temp != nullptr) {
419             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
420             temp = temp->next;
421         }
422         return sum;
423     }
424
425     // 獲取堆疊所有元素的平方根的倒數平方和
426     T sumOfReciprocalsOfSqrtOfSquares() const {
427         T sum = 0;
428         Node<T>* temp = topNode;
429         while (temp != nullptr) {
430             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
431             temp = temp->next;
432         }
433         return sum;
434     }
435
436     // 獲取堆疊所有元素的平方根的倒數平方和
437     T sumOfReciprocalsOfSqrtOfSquares() const {
438         T sum = 0;
439         Node<T>* temp = topNode;
440         while (temp != nullptr) {
441             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
442             temp = temp->next;
443         }
444         return sum;
445     }
446
447     // 獲取堆疊所有元素的平方根的倒數平方和
448     T sumOfReciprocalsOfSqrtOfSquares() const {
449         T sum = 0;
450         Node<T>* temp = topNode;
451         while (temp != nullptr) {
452             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
453             temp = temp->next;
454         }
455         return sum;
456     }
457
458     // 獲取堆疊所有元素的平方根的倒數平方和
459     T sumOfReciprocalsOfSqrtOfSquares() const {
460         T sum = 0;
461         Node<T>* temp = topNode;
462         while (temp != nullptr) {
463             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
464             temp = temp->next;
465         }
466         return sum;
467     }
468
469     // 獲取堆疊所有元素的平方根的倒數平方和
470     T sumOfReciprocalsOfSqrtOfSquares() const {
471         T sum = 0;
472         Node<T>* temp = topNode;
473         while (temp != nullptr) {
474             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
475             temp = temp->next;
476         }
477         return sum;
478     }
479
480     // 獲取堆疊所有元素的平方根的倒數平方和
481     T sumOfReciprocalsOfSqrtOfSquares() const {
482         T sum = 0;
483         Node<T>* temp = topNode;
484         while (temp != nullptr) {
485             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
486             temp = temp->next;
487         }
488         return sum;
489     }
490
491     // 獲取堆疊所有元素的平方根的倒數平方和
492     T sumOfReciprocalsOfSqrtOfSquares() const {
493         T sum = 0;
494         Node<T>* temp = topNode;
495         while (temp != nullptr) {
496             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
497             temp = temp->next;
498         }
499         return sum;
500     }
501
502     // 獲取堆疊所有元素的平方根的倒數平方和
503     T sumOfReciprocalsOfSqrtOfSquares() const {
504         T sum = 0;
505         Node<T>* temp = topNode;
506         while (temp != nullptr) {
507             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
508             temp = temp->next;
509         }
510         return sum;
511     }
512
513     // 獲取堆疊所有元素的平方根的倒數平方和
514     T sumOfReciprocalsOfSqrtOfSquares() const {
515         T sum = 0;
516         Node<T>* temp = topNode;
517         while (temp != nullptr) {
518             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
519             temp = temp->next;
520         }
521         return sum;
522     }
523
524     // 獲取堆疊所有元素的平方根的倒數平方和
525     T sumOfReciprocalsOfSqrtOfSquares() const {
526         T sum = 0;
527         Node<T>* temp = topNode;
528         while (temp != nullptr) {
529             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
530             temp = temp->next;
531         }
532         return sum;
533     }
534
535     // 獲取堆疊所有元素的平方根的倒數平方和
536     T sumOfReciprocalsOfSqrtOfSquares() const {
537         T sum = 0;
538         Node<T>* temp = topNode;
539         while (temp != nullptr) {
540             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
541             temp = temp->next;
542         }
543         return sum;
544     }
545
546     // 獲取堆疊所有元素的平方根的倒數平方和
547     T sumOfReciprocalsOfSqrtOfSquares() const {
548         T sum = 0;
549         Node<T>* temp = topNode;
550         while (temp != nullptr) {
551             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
552             temp = temp->next;
553         }
554         return sum;
555     }
556
557     // 獲取堆疊所有元素的平方根的倒數平方和
558     T sumOfReciprocalsOfSqrtOfSquares() const {
559         T sum = 0;
560         Node<T>* temp = topNode;
561         while (temp != nullptr) {
562             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
563             temp = temp->next;
564         }
565         return sum;
566     }
567
568     // 獲取堆疊所有元素的平方根的倒數平方和
569     T sumOfReciprocalsOfSqrtOfSquares() const {
570         T sum = 0;
571         Node<T>* temp = topNode;
572         while (temp != nullptr) {
573             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
574             temp = temp->next;
575         }
576         return sum;
577     }
578
579     // 獲取堆疊所有元素的平方根的倒數平方和
580     T sumOfReciprocalsOfSqrtOfSquares() const {
581         T sum = 0;
582         Node<T>* temp = topNode;
583         while (temp != nullptr) {
584             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
585             temp = temp->next;
586         }
587         return sum;
588     }
589
590     // 獲取堆疊所有元素的平方根的倒數平方和
591     T sumOfReciprocalsOfSqrtOfSquares() const {
592         T sum = 0;
593         Node<T>* temp = topNode;
594         while (temp != nullptr) {
595             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
596             temp = temp->next;
597         }
598         return sum;
599     }
600
601     // 獲取堆疊所有元素的平方根的倒數平方和
602     T sumOfReciprocalsOfSqrtOfSquares() const {
603         T sum = 0;
604         Node<T>* temp = topNode;
605         while (temp != nullptr) {
606             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
607             temp = temp->next;
608         }
609         return sum;
610     }
611
612     // 獲取堆疊所有元素的平方根的倒數平方和
613     T sumOfReciprocalsOfSqrtOfSquares() const {
614         T sum = 0;
615         Node<T>* temp = topNode;
616         while (temp != nullptr) {
617             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
618             temp = temp->next;
619         }
620         return sum;
621     }
622
623     // 獲取堆疊所有元素的平方根的倒數平方和
624     T sumOfReciprocalsOfSqrtOfSquares() const {
625         T sum = 0;
626         Node<T>* temp = topNode;
627         while (temp != nullptr) {
628             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
629             temp = temp->next;
630         }
631         return sum;
632     }
633
634     // 獲取堆疊所有元素的平方根的倒數平方和
635     T sumOfReciprocalsOfSqrtOfSquares() const {
636         T sum = 0;
637         Node<T>* temp = topNode;
638         while (temp != nullptr) {
639             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
640             temp = temp->next;
641         }
642         return sum;
643     }
644
645     // 獲取堆疊所有元素的平方根的倒數平方和
646     T sumOfReciprocalsOfSqrtOfSquares() const {
647         T sum = 0;
648         Node<T>* temp = topNode;
649         while (temp != nullptr) {
650             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
651             temp = temp->next;
652         }
653         return sum;
654     }
655
656     // 獲取堆疊所有元素的平方根的倒數平方和
657     T sumOfReciprocalsOfSqrtOfSquares() const {
658         T sum = 0;
659         Node<T>* temp = topNode;
660         while (temp != nullptr) {
661             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
662             temp = temp->next;
663         }
664         return sum;
665     }
666
667     // 獲取堆疊所有元素的平方根的倒數平方和
668     T sumOfReciprocalsOfSqrtOfSquares() const {
669         T sum = 0;
670         Node<T>* temp = topNode;
671         while (temp != nullptr) {
672             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
673             temp = temp->next;
674         }
675         return sum;
676     }
677
678     // 獲取堆疊所有元素的平方根的倒數平方和
679     T sumOfReciprocalsOfSqrtOfSquares() const {
680         T sum = 0;
681         Node<T>* temp = topNode;
682         while (temp != nullptr) {
683             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
684             temp = temp->next;
685         }
686         return sum;
687     }
688
689     // 獲取堆疊所有元素的平方根的倒數平方和
690     T sumOfReciprocalsOfSqrtOfSquares() const {
691         T sum = 0;
692         Node<T>* temp = topNode;
693         while (temp != nullptr) {
694             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
695             temp = temp->next;
696         }
697         return sum;
698     }
699
700     // 獲取堆疊所有元素的平方根的倒數平方和
701     T sumOfReciprocalsOfSqrtOfSquares() const {
702         T sum = 0;
703         Node<T>* temp = topNode;
704         while (temp != nullptr) {
705             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
706             temp = temp->next;
707         }
708         return sum;
709     }
710
711     // 獲取堆疊所有元素的平方根的倒數平方和
712     T sumOfReciprocalsOfSqrtOfSquares() const {
713         T sum = 0;
714         Node<T>* temp = topNode;
715         while (temp != nullptr) {
716             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
717             temp = temp->next;
718         }
719         return sum;
720     }
721
722     // 獲取堆疊所有元素的平方根的倒數平方和
723     T sumOfReciprocalsOfSqrtOfSquares() const {
724         T sum = 0;
725         Node<T>* temp = topNode;
726         while (temp != nullptr) {
727             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
728             temp = temp->next;
729         }
730         return sum;
731     }
732
733     // 獲取堆疊所有元素的平方根的倒數平方和
734     T sumOfReciprocalsOfSqrtOfSquares() const {
735         T sum = 0;
736         Node<T>* temp = topNode;
737         while (temp != nullptr) {
738             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
739             temp = temp->next;
740         }
741         return sum;
742     }
743
744     // 獲取堆疊所有元素的平方根的倒數平方和
745     T sumOfReciprocalsOfSqrtOfSquares() const {
746         T sum = 0;
747         Node<T>* temp = topNode;
748         while (temp != nullptr) {
749             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
750             temp = temp->next;
751         }
752         return sum;
753     }
754
755     // 獲取堆疊所有元素的平方根的倒數平方和
756     T sumOfReciprocalsOfSqrtOfSquares() const {
757         T sum = 0;
758         Node<T>* temp = topNode;
759         while (temp != nullptr) {
760             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
761             temp = temp->next;
762         }
763         return sum;
764     }
765
766     // 獲取堆疊所有元素的平方根的倒數平方和
767     T sumOfReciprocalsOfSqrtOfSquares() const {
768         T sum = 0;
769         Node<T>* temp = topNode;
770         while (temp != nullptr) {
771             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
772             temp = temp->next;
773         }
774         return sum;
775     }
776
777     // 獲取堆疊所有元素的平方根的倒數平方和
778     T sumOfReciprocalsOfSqrtOfSquares() const {
779         T sum = 0;
780         Node<T>* temp = topNode;
781         while (temp != nullptr) {
782             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
783             temp = temp->next;
784         }
785         return sum;
786     }
787
788     // 獲取堆疊所有元素的平方根的倒數平方和
789     T sumOfReciprocalsOfSqrtOfSquares() const {
790         T sum = 0;
791         Node<T>* temp = topNode;
792         while (temp != nullptr) {
793             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
794             temp = temp->next;
795         }
796         return sum;
797     }
798
799     // 獲取堆疊所有元素的平方根的倒數平方和
800     T sumOfReciprocalsOfSqrtOfSquares() const {
801         T sum = 0;
802         Node<T>* temp = topNode;
803         while (temp != nullptr) {
804             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
805             temp = temp->next;
806         }
807         return sum;
808     }
809
810     // 獲取堆疊所有元素的平方根的倒數平方和
811     T sumOfReciprocalsOfSqrtOfSquares() const {
812         T sum = 0;
813         Node<T>* temp = topNode;
814         while (temp != nullptr) {
815             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
816             temp = temp->next;
817         }
818         return sum;
819     }
820
821     // 獲取堆疊所有元素的平方根的倒數平方和
822     T sumOfReciprocalsOfSqrtOfSquares() const {
823         T sum = 0;
824         Node<T>* temp = topNode;
825         while (temp != nullptr) {
826             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
827             temp = temp->next;
828         }
829         return sum;
830     }
831
832     // 獲取堆疊所有元素的平方根的倒數平方和
833     T sumOfReciprocalsOfSqrtOfSquares() const {
834         T sum = 0;
835         Node<T>* temp = topNode;
836         while (temp != nullptr) {
837             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
838             temp = temp->next;
839         }
840         return sum;
841     }
842
843     // 獲取堆疊所有元素的平方根的倒數平方和
844     T sumOfReciprocalsOfSqrtOfSquares() const {
845         T sum = 0;
846         Node<T>* temp = topNode;
847         while (temp != nullptr) {
848             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
849             temp = temp->next;
850         }
851         return sum;
852     }
853
854     // 獲取堆疊所有元素的平方根的倒數平方和
855     T sumOfReciprocalsOfSqrtOfSquares() const {
856         T sum = 0;
857         Node<T>* temp = topNode;
858         while (temp != nullptr) {
859             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
860             temp = temp->next;
861         }
862         return sum;
863     }
864
865     // 獲取堆疊所有元素的平方根的倒數平方和
866     T sumOfReciprocalsOfSqrtOfSquares() const {
867         T sum = 0;
868         Node<T>* temp = topNode;
869         while (temp != nullptr) {
870             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
871             temp = temp->next;
872         }
873         return sum;
874     }
875
876     // 獲取堆疊所有元素的平方根的倒數平方和
877     T sumOfReciprocalsOfSqrtOfSquares() const {
878         T sum = 0;
879         Node<T>* temp = topNode;
880         while (temp != nullptr) {
881             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
882             temp = temp->next;
883         }
884         return sum;
885     }
886
887     // 獲取堆疊所有元素的平方根的倒數平方和
888     T sumOfReciprocalsOfSqrtOfSquares() const {
889         T sum = 0;
890         Node<T>* temp = topNode;
891         while (temp != nullptr) {
892             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
893             temp = temp->next;
894         }
895         return sum;
896     }
897
898     // 獲取堆疊所有元素的平方根的倒數平方和
899     T sumOfReciprocalsOfSqrtOfSquares() const {
900         T sum = 0;
901         Node<T>* temp = topNode;
902         while (temp != nullptr) {
903             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
904             temp = temp->next;
905         }
906         return sum;
907     }
908
909     // 獲取堆疊所有元素的平方根的倒數平方和
910     T sumOfReciprocalsOfSqrtOfSquares() const {
911         T sum = 0;
912         Node<T>* temp = topNode;
913         while (temp != nullptr) {
914             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
915             temp = temp->next;
916         }
917         return sum;
918     }
919
920     // 獲取堆疊所有元素的平方根的倒數平方和
921     T sumOfReciprocalsOfSqrtOfSquares() const {
922         T sum = 0;
923         Node<T>* temp = topNode;
924         while (temp != nullptr) {
925             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
926             temp = temp->next;
927         }
928         return sum;
929     }
930
931     // 獲取堆疊所有元素的平方根的倒數平方和
932     T sumOfReciprocalsOfSqrtOfSquares() const {
933         T sum = 0;
934         Node<T>* temp = topNode;
935         while (temp != nullptr) {
936             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
937             temp = temp->next;
938         }
939         return sum;
940     }
941
942     // 獲取堆疊所有元素的平方根的倒數平方和
943     T sumOfReciprocalsOfSqrtOfSquares() const {
944         T sum = 0;
945         Node<T>* temp = topNode;
946         while (temp != nullptr) {
947             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
948             temp = temp->next;
949         }
950         return sum;
951     }
952
953     // 獲取堆疊所有元素的平方根的倒數平方和
954     T sumOfReciprocalsOfSqrtOfSquares() const {
955         T sum = 0;
956         Node<T>* temp = topNode;
957         while (temp != nullptr) {
958             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
959             temp = temp->next;
960         }
961         return sum;
962     }
963
964     // 獲取堆疊所有元素的平方根的倒數平方和
965     T sumOfReciprocalsOfSqrtOfSquares() const {
966         T sum = 0;
967         Node<T>* temp = topNode;
968         while (temp != nullptr) {
969             sum += 1 / (sqrt(temp->data) * sqrt(temp->data));
970             temp = temp->next;
971         }
972         return sum;
973     }
974
975     // 獲取堆疊所有元素的平方根的倒數平方和
976     T sumOfReciprocalsOfSqrtOfSquares() const {
977         T sum = 0;
978         Node<T>*
```

輸入 2^3+1 ，輸出 9.0

The screenshot shows a C++ IDE with a file named `l.cpp`. The code implements a linked list structure using templates. It includes headers for `<iostream>`, `<string>`, `<vector>`, `<cmath>`, `<stack>`, and `<ctype>`. The `Node` struct contains a `T data` member and a `Node* next` pointer. The `LinkedListStack` class uses a stack to manage the nodes. The `main` function prompts the user to input a formula, which is then evaluated using the linked list structure. The output shows the result of the calculation 2^3+1 as 9.0.

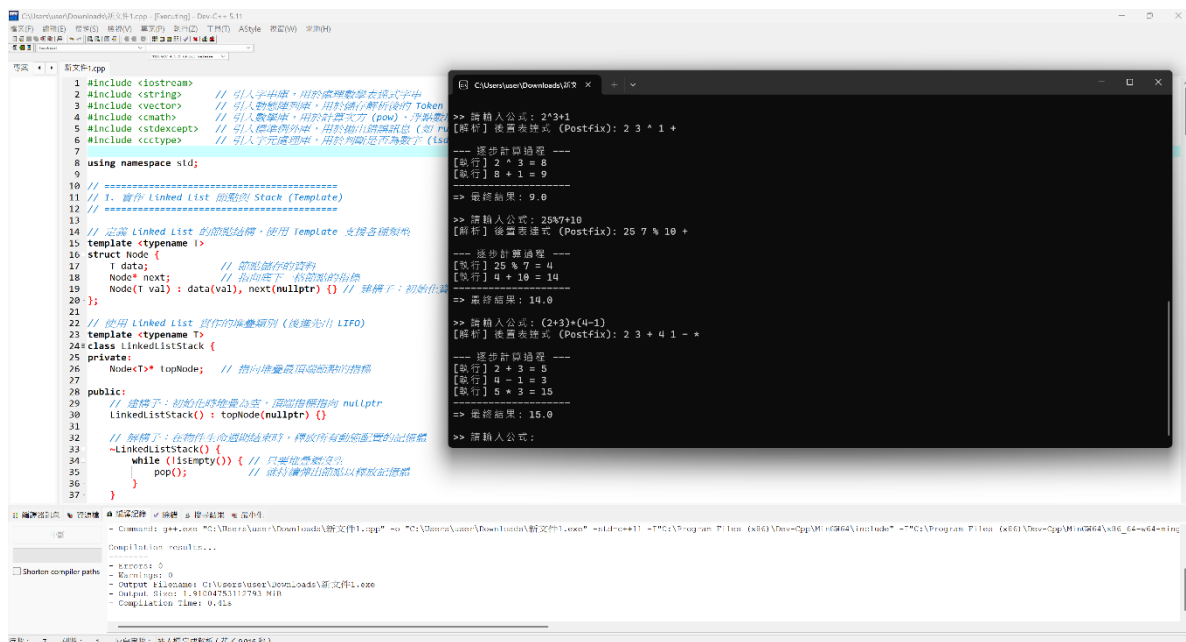
```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <cmath>
5 #include <stack>
6 #include <ctype>
7
8 using namespace std;
9
10 // *****
11 // 1. 實作 Linked List 節點與 Stack (Template)
12 // *****
13
14 // 定義 Linked List 的節點結構，使用 Template 支援各種類型
15 template <typename T>
16 struct Node {
17     T data; // 節點儲存的資料
18     Node* next; // 指向下一個節點的指標
19     Node(T val) : data(val), next(nullptr) {} // 建構子：初始化
20 };
21
22 // 使用 Linked List 實作的堆疊類別 (後進先出 LIFO)
23 template <typename T>
24 class LinkedListStack {
25 private:
26     Node<T*> topNode; // 指向堆疊最頂端節點的指標
27
28 public:
29     // 建構子：初始化時堆疊為空，頂端指標指向 nullptr
30     LinkedListStack() : topNode(nullptr) {}
31
32     // 解構子：在所有生命週期結束時，釋放所有動態配置的記憶體
33     ~LinkedListStack() {
34         while (!isEmpty()) { // 只要堆疊還沒空
35             pop(); // 就持續彈出節點以釋放記憶體
36         }
37     }
38
39     // 其他方法...
40 };
41
42 int main() {
43     // ...
44 }
```

輸入 $25\%7+10$ ，輸出 14.0

The screenshot shows the same C++ IDE with the `l.cpp` file. The code is identical to the previous one, but the execution results show the calculation of $25\%7+10$, resulting in 14.0. The output window displays the input formula, the postfix expression `25 7 % 10 +`, and the final result 14.0.

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <cmath>
5 #include <stack>
6 #include <ctype>
7
8 using namespace std;
9
10 // *****
11 // 1. 實作 Linked List 節點與 Stack (Template)
12 // *****
13
14 // 定義 Linked List 的節點結構，使用 Template 支援各種類型
15 template <typename T>
16 struct Node {
17     T data; // 節點儲存的資料
18     Node* next; // 指向下一個節點的指標
19     Node(T val) : data(val), next(nullptr) {} // 建構子：初始化
20 };
21
22 // 使用 Linked List 實作的堆疊類別 (後進先出 LIFO)
23 template <typename T>
24 class LinkedListStack {
25 private:
26     Node<T*> topNode; // 指向堆疊最頂端節點的指標
27
28 public:
29     // 建構子：初始化時堆疊為空，頂端指標指向 nullptr
30     LinkedListStack() : topNode(nullptr) {}
31
32     // 解構子：在所有生命週期結束時，釋放所有動態配置的記憶體
33     ~LinkedListStack() {
34         while (!isEmpty()) { // 只要堆疊還沒空
35             pop(); // 就持續彈出節點以釋放記憶體
36         }
37     }
38
39     // 其他方法...
40 };
41
42 int main() {
43     // ...
44 }
```

輸入 $(2+3)*(4-1)$ ，輸出 15.0



The screenshot shows a C++ IDE with a file named `linklist1.cpp`. The code implements a linked list stack using a template. It includes headers for `iostream`, `string`, `vector`, `cmath`, `stack`, and `ctype`. The `Node` struct contains a `data` member and a `next` pointer. The `LinkedListStack` class uses a `topNode` pointer and a `LinkedList` object. The `main` function is not visible in the snippet.

Overlaid on the IDE is a calculator window titled `C:\Users\user\Downloads\linklist1.exe`. It shows the following calculations:

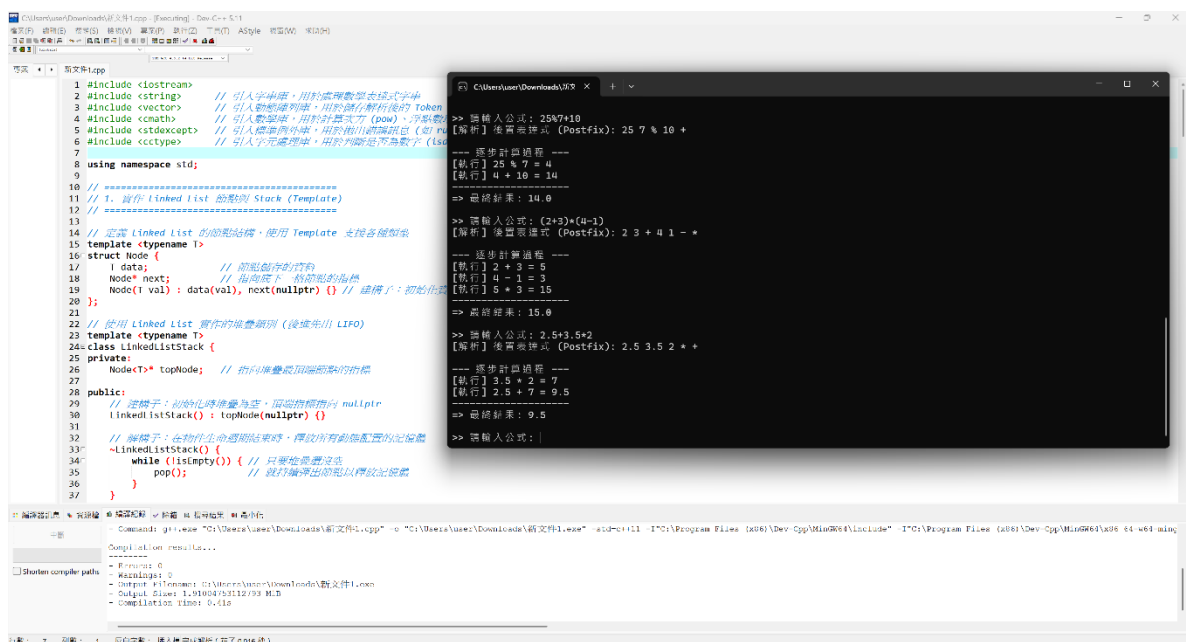
```
>> 輸入公式: 2+3*1
【解析】後置表達式 (Postfix): 2 3 * 1 +
--- 逐步計算過程 ---
【執行】2 * 3 = 6
【執行】6 + 1 = 7
=> 最終結果: 7.0

>> 輸入公式: 25%7+10
【解析】後置表達式 (Postfix): 25 7 % 10 +
--- 逐步計算過程 ---
【執行】25 % 7 = 4
【執行】4 + 10 = 14
=> 最終結果: 14.0

>> 輸入公式: (2+3)*(4-1)
【解析】後置表達式 (Postfix): 2 3 + 4 1 - *
--- 逐步計算過程 ---
【執行】2 + 3 = 5
【執行】4 - 1 = 3
【執行】5 * 3 = 15
=> 最終結果: 15.0

>> 輸入公式:
```

輸入 $2.5+3.5*2$ ，輸出 9.5



The screenshot shows the same C++ IDE with the `linklist1.cpp` file. The code is identical to the previous screenshot. The calculator window shows the following calculations:

```
>> 輸入公式: 25%7+10
【解析】後置表達式 (Postfix): 25 7 % 10 +
--- 逐步計算過程 ---
【執行】25 % 7 = 4
【執行】4 + 10 = 14
=> 最終結果: 14.0

>> 輸入公式: (2+3)*(4-1)
【解析】後置表達式 (Postfix): 2 3 + 4 1 - *
--- 逐步計算過程 ---
【執行】2 + 3 = 5
【執行】4 - 1 = 3
【執行】5 * 3 = 15
=> 最終結果: 15.0

>> 輸入公式: 2.5+3.5*2
【解析】後置表達式 (Postfix): 2.5 3.5 2 * +
--- 逐步計算過程 ---
【執行】3.5 * 2 = 7
【執行】2.5 + 7 = 9.5
=> 最終結果: 9.5

>> 輸入公式:
```

若輸入亂碼，輸出 Error

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <cmath>
5 #include <stdexcept>
6 #include <ctype>
7
8 using namespace std;
9
10 // =====
11 // 1. 實作 Linked List 節點與 Stack (Template)
12 // =====
13
14 // 定義 Linked List 的節點結構，使用 Template 以支援不同資料型態
15 template <typename T>
16 struct Node {
17     T data; // 節點儲存的資料
18     Node* next; // 指向下一個節點的指標
19     // 初始化時，next 設為 nullptr
20     Node(T val) : data(val), next(nullptr) {}
21 };
22
23 // 使用 Linked List 實作你的堆疊 (Stack) 類別
24 template <typename T>
25 class LinkedListStack {
26 private:
27     Node<T>* topNode; // 指向堆疊最頂端節點的指標
28
29 public:
30     // 初始化時，堆疊為空
31     LinkedListStack() : topNode(nullptr) {}
32
33     // 建構子：當堆疊被創建時，釋放所有動態分配的記憶體
34     ~LinkedListStack() {
35         while (!isEmpty()) {
36             pop();
37         }
38     }
39
40     // 判斷堆疊是否為空
41     bool isEmpty() const {
42         return topNode == nullptr;
43     }
44
45     // 將元素推入堆疊
46     void push(T val) {
47         Node<T>* newNode = new Node<T>(val);
48         newNode->next = topNode;
49         topNode = newNode;
50     }
51
52     // 從堆疊中移除元素
53     void pop() {
54         if (!isEmpty()) {
55             Node<T>* temp = topNode;
56             topNode = temp->next;
57             delete temp;
58         }
59     }
60
61     // 顯示堆疊內容
62     void display() const {
63         if (isEmpty()) {
64             cout << "Stack is empty." << endl;
65         } else {
66             Node<T>* temp = topNode;
67             while (temp != nullptr) {
68                 cout << temp->data << " ";
69                 temp = temp->next;
70             }
71             cout << endl;
72         }
73     }
74
75     // 計算堆疊大小
76     int size() const {
77         int count = 0;
78         Node<T>* temp = topNode;
79         while (temp != nullptr) {
80             count++;
81             temp = temp->next;
82         }
83         return count;
84     }
85
86     // 清空堆疊
87     void clear() {
88         while (!isEmpty()) {
89             pop();
90         }
91     }
92
93     // 獲取堆疊頂端元素
94     T top() const {
95         if (!isEmpty()) {
96             return topNode->data;
97         }
98         throw std::runtime_error("Stack is empty, cannot get top element.");
99     }
100
101     // 獲取堆疊底端元素
102     T bottom() const {
103         if (!isEmpty()) {
104             Node<T>* temp = topNode;
105             while (temp->next != nullptr) {
106                 temp = temp->next;
107             }
108             return temp->data;
109         }
110         throw std::runtime_error("Stack is empty, cannot get bottom element.");
111     }
112
113     // 獲取堆疊所有元素
114     vector<T> toVector() const {
115         vector<T> v;
116         Node<T>* temp = topNode;
117         while (temp != nullptr) {
118             v.push_back(temp->data);
119             temp = temp->next;
120         }
121         return v;
122     }
123
124     // 獲取堆疊所有元素的平方和
125     double sumOfSquares() const {
126         double sum = 0;
127         Node<T>* temp = topNode;
128         while (temp != nullptr) {
129             sum += temp->data * temp->data;
130             temp = temp->next;
131         }
132         return sum;
133     }
134
135     // 獲取堆疊所有元素的平均值
136     double average() const {
137         if (size() > 0) {
138             return sumOfSquares() / size();
139         }
140         throw std::runtime_error("Stack is empty, cannot calculate average.");
141     }
142
143     // 獲取堆疊所有元素的標準差
144     double standardDeviation() const {
145         if (size() > 0) {
146             double avg = average();
147             double sumOfSquares = sumOfSquares();
148             double variance = (sumOfSquares / size() - avg * avg);
149             return sqrt(variance);
150         }
151         throw std::runtime_error("Stack is empty, cannot calculate standard deviation.");
152     }
153
154     // 獲取堆疊所有元素的方差
155     double variance() const {
156         if (size() > 0) {
157             double avg = average();
158             double sumOfSquares = sumOfSquares();
159             return (sumOfSquares / size() - avg * avg);
160         }
161         throw std::runtime_error("Stack is empty, cannot calculate variance.");
162     }
163
164     // 獲取堆疊所有元素的協方差
165     double covariance() const {
166         if (size() > 0) {
167             double avg = average();
168             double sumOfProducts = 0;
169             Node<T>* temp = topNode;
170             while (temp != nullptr) {
171                 sumOfProducts += temp->data * (temp->data - avg);
172                 temp = temp->next;
173             }
174             return sumOfProducts / size();
175         }
176         throw std::runtime_error("Stack is empty, cannot calculate covariance.");
177     }
178
179     // 獲取堆疊所有元素的相关系数
180     double correlationCoefficient() const {
181         if (size() > 0) {
182             double avg = average();
183             double sumOfProducts = 0;
184             Node<T>* temp = topNode;
185             while (temp != nullptr) {
186                 sumOfProducts += temp->data * (temp->data - avg);
187                 temp = temp->next;
188             }
189             double variance = variance();
190             return sumOfProducts / (size() * variance);
191         }
192         throw std::runtime_error("Stack is empty, cannot calculate correlation coefficient.");
193     }
194
195     // 獲取堆疊所有元素的皮爾森相关系数
196     double pearsonCorrelationCoefficient() const {
197         if (size() > 0) {
198             double avg = average();
199             double sumOfProducts = 0;
200             Node<T>* temp = topNode;
201             while (temp != nullptr) {
202                 sumOfProducts += temp->data * (temp->data - avg);
203                 temp = temp->next;
204             }
205             double variance = variance();
206             return sumOfProducts / (size() * variance);
207         }
208         throw std::runtime_error("Stack is empty, cannot calculate pearson correlation coefficient.");
209     }
210
211     // 獲取堆疊所有元素的斯皮爾曼相关系数
212     double spearmanCorrelationCoefficient() const {
213         if (size() > 0) {
214             double avg = average();
215             double sumOfProducts = 0;
216             Node<T>* temp = topNode;
217             while (temp != nullptr) {
218                 sumOfProducts += temp->data * (temp->data - avg);
219                 temp = temp->next;
220             }
221             double variance = variance();
222             return sumOfProducts / (size() * variance);
223         }
224         throw std::runtime_error("Stack is empty, cannot calculate spearman correlation coefficient.");
225     }
226
227     // 獲取堆疊所有元素的肯德尔相关系数
228     double kendallCorrelationCoefficient() const {
229         if (size() > 0) {
230             double avg = average();
231             double sumOfProducts = 0;
232             Node<T>* temp = topNode;
233             while (temp != nullptr) {
234                 sumOfProducts += temp->data * (temp->data - avg);
235                 temp = temp->next;
236             }
237             double variance = variance();
238             return sumOfProducts / (size() * variance);
239         }
240         throw std::runtime_error("Stack is empty, cannot calculate kendall correlation coefficient.");
241     }
242
243     // 獲取堆疊所有元素的曼哈頓距离
244     double manhattanDistance() const {
245         if (size() > 0) {
246             double sumOfAbsoluteDifferences = 0;
247             Node<T>* temp = topNode;
248             while (temp != nullptr) {
249                 sumOfAbsoluteDifferences += temp->data;
250                 temp = temp->next;
251             }
252             return sumOfAbsoluteDifferences;
253         }
254         throw std::runtime_error("Stack is empty, cannot calculate manhattan distance.");
255     }
256
257     // 獲取堆疊所有元素的歐幾里得距离
258     double euclideanDistance() const {
259         if (size() > 0) {
260             double sumOfSquares = sumOfSquares();
261             return sqrt(sumOfSquares);
262         }
263         throw std::runtime_error("Stack is empty, cannot calculate euclidean distance.");
264     }
265
266     // 獲取堆疊所有元素的切比雪夫距离
267     double chebyshevDistance() const {
268         if (size() > 0) {
269             double maxAbsoluteDifference = 0;
270             Node<T>* temp = topNode;
271             while (temp != nullptr) {
272                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
273                 temp = temp->next;
274             }
275             return maxAbsoluteDifference;
276         }
277         throw std::runtime_error("Stack is empty, cannot calculate chebyshev distance.");
278     }
279
280     // 獲取堆疊所有元素的曼哈頓距离的平方和
281     double sumOfManhattanDistances() const {
282         if (size() > 0) {
283             double sumOfAbsoluteDifferences = 0;
284             Node<T>* temp = topNode;
285             while (temp != nullptr) {
286                 sumOfAbsoluteDifferences += temp->data;
287                 temp = temp->next;
288             }
289             return sumOfAbsoluteDifferences * sumOfAbsoluteDifferences;
290         }
291         throw std::runtime_error("Stack is empty, cannot calculate sum of manhattan distances.");
292     }
293
294     // 獲取堆疊所有元素的歐幾里得距离的平方和
295     double sumOfEuclideanDistances() const {
296         if (size() > 0) {
297             double sumOfSquares = sumOfSquares();
298             return sumOfSquares * sumOfSquares;
299         }
300         throw std::runtime_error("Stack is empty, cannot calculate sum of euclidean distances.");
301     }
302
303     // 獲取堆疊所有元素的切比雪夫距离的平方和
304     double sumOfChebyshevDistances() const {
305         if (size() > 0) {
306             double maxAbsoluteDifference = 0;
307             Node<T>* temp = topNode;
308             while (temp != nullptr) {
309                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
310                 temp = temp->next;
311             }
312             return maxAbsoluteDifference * maxAbsoluteDifference;
313         }
314         throw std::runtime_error("Stack is empty, cannot calculate sum of chebyshev distances.");
315     }
316
317     // 獲取堆疊所有元素的曼哈頓距离的平均值
318     double averageManhattanDistance() const {
319         if (size() > 0) {
320             double sumOfAbsoluteDifferences = 0;
321             Node<T>* temp = topNode;
322             while (temp != nullptr) {
323                 sumOfAbsoluteDifferences += temp->data;
324                 temp = temp->next;
325             }
326             return sumOfAbsoluteDifferences / size();
327         }
328         throw std::runtime_error("Stack is empty, cannot calculate average manhattan distance.");
329     }
330
331     // 獲取堆疊所有元素的歐幾里得距离的平均值
332     double averageEuclideanDistance() const {
333         if (size() > 0) {
334             double sumOfSquares = sumOfSquares();
335             return sqrt(sumOfSquares) / size();
336         }
337         throw std::runtime_error("Stack is empty, cannot calculate average euclidean distance.");
338     }
339
340     // 獲取堆疊所有元素的切比雪夫距离的平均值
341     double averageChebyshevDistance() const {
342         if (size() > 0) {
343             double maxAbsoluteDifference = 0;
344             Node<T>* temp = topNode;
345             while (temp != nullptr) {
346                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
347                 temp = temp->next;
348             }
349             return maxAbsoluteDifference / size();
350         }
351         throw std::runtime_error("Stack is empty, cannot calculate average chebyshev distance.");
352     }
353
354     // 獲取堆疊所有元素的曼哈頓距离的平方和的平均值
355     double averageSumOfManhattanDistances() const {
356         if (size() > 0) {
357             double sumOfAbsoluteDifferences = 0;
358             Node<T>* temp = topNode;
359             while (temp != nullptr) {
360                 sumOfAbsoluteDifferences += temp->data;
361                 temp = temp->next;
362             }
363             return (sumOfAbsoluteDifferences * sumOfAbsoluteDifferences) / size();
364         }
365         throw std::runtime_error("Stack is empty, cannot calculate average sum of manhattan distances.");
366     }
367
368     // 獲取堆疊所有元素的歐幾里得距离的平方和的平均值
369     double averageSumOfEuclideanDistances() const {
370         if (size() > 0) {
371             double sumOfSquares = sumOfSquares();
372             return (sumOfSquares * sumOfSquares) / size();
373         }
374         throw std::runtime_error("Stack is empty, cannot calculate average sum of euclidean distances.");
375     }
376
377     // 獲取堆疊所有元素的切比雪夫距离的平方和的平均值
378     double averageSumOfChebyshevDistances() const {
379         if (size() > 0) {
380             double maxAbsoluteDifference = 0;
381             Node<T>* temp = topNode;
382             while (temp != nullptr) {
383                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
384                 temp = temp->next;
385             }
386             return (maxAbsoluteDifference * maxAbsoluteDifference) / size();
387         }
388         throw std::runtime_error("Stack is empty, cannot calculate average sum of chebyshev distances.");
389     }
390
391     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和
392     double sumOfAverageManhattanDistances() const {
393         if (size() > 0) {
394             double sumOfAbsoluteDifferences = 0;
395             Node<T>* temp = topNode;
396             while (temp != nullptr) {
397                 sumOfAbsoluteDifferences += temp->data;
398                 temp = temp->next;
399             }
400             return (sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size());
401         }
402         throw std::runtime_error("Stack is empty, cannot calculate sum of average manhattan distances.");
403     }
404
405     // 獲取堆疊所有元素的歐幾里得距离的平均值的平方和
406     double sumOfAverageEuclideanDistances() const {
407         if (size() > 0) {
408             double sumOfSquares = sumOfSquares();
409             return (sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size());
410         }
411         throw std::runtime_error("Stack is empty, cannot calculate sum of average euclidean distances.");
412     }
413
414     // 獲取堆疊所有元素的切比雪夫距离的平均值的平方和
415     double sumOfAverageChebyshevDistances() const {
416         if (size() > 0) {
417             double maxAbsoluteDifference = 0;
418             Node<T>* temp = topNode;
419             while (temp != nullptr) {
420                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
421                 temp = temp->next;
422             }
423             return (maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size());
424         }
425         throw std::runtime_error("Stack is empty, cannot calculate sum of average chebyshev distances.");
426     }
427
428     // 獲取堆疊所有元素的曼哈頓距离的平方和的平均值的平方和
429     double sumOfAverageSumOfManhattanDistances() const {
430         if (size() > 0) {
431             double sumOfAbsoluteDifferences = 0;
432             Node<T>* temp = topNode;
433             while (temp != nullptr) {
434                 sumOfAbsoluteDifferences += temp->data;
435                 temp = temp->next;
436             }
437             return ((sumOfAbsoluteDifferences * sumOfAbsoluteDifferences) / size()) * ((sumOfAbsoluteDifferences * sumOfAbsoluteDifferences) / size());
438         }
439         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of manhattan distances.");
440     }
441
442     // 獲取堆疊所有元素的歐幾里得距离的平方和的平均值的平方和
443     double sumOfAverageSumOfEuclideanDistances() const {
444         if (size() > 0) {
445             double sumOfSquares = sumOfSquares();
446             return ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size()));
447         }
448         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of euclidean distances.");
449     }
450
451     // 獲取堆疊所有元素的切比雪夫距离的平方和的平均值的平方和
452     double sumOfAverageSumOfChebyshevDistances() const {
453         if (size() > 0) {
454             double maxAbsoluteDifference = 0;
455             Node<T>* temp = topNode;
456             while (temp != nullptr) {
457                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
458                 temp = temp->next;
459             }
460             return ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size()));
461         }
462         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of chebyshev distances.");
463     }
464
465     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和的平均值
466     double averageSumOfAverageManhattanDistances() const {
467         if (size() > 0) {
468             double sumOfAbsoluteDifferences = 0;
469             Node<T>* temp = topNode;
470             while (temp != nullptr) {
471                 sumOfAbsoluteDifferences += temp->data;
472                 temp = temp->next;
473             }
474             return ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) / size();
475         }
476         throw std::runtime_error("Stack is empty, cannot calculate average sum of average manhattan distances.");
477     }
478
479     // 獲取堆疊所有元素的歐幾里得距离的平均值的平方和的平均值
480     double averageSumOfAverageEuclideanDistances() const {
481         if (size() > 0) {
482             double sumOfSquares = sumOfSquares();
483             return ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) / size();
484         }
485         throw std::runtime_error("Stack is empty, cannot calculate average sum of average euclidean distances.");
486     }
487
488     // 獲取堆疊所有元素的切比雪夫距离的平均值的平方和的平均值
489     double averageSumOfAverageChebyshevDistances() const {
490         if (size() > 0) {
491             double maxAbsoluteDifference = 0;
492             Node<T>* temp = topNode;
493             while (temp != nullptr) {
494                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
495                 temp = temp->next;
496             }
497             return ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) / size();
498         }
499         throw std::runtime_error("Stack is empty, cannot calculate average sum of average chebyshev distances.");
500     }
501
502     // 獲取堆疊所有元素的曼哈頓距离的平方和的平均值的平方和的平均值
503     double averageSumOfAverageSumOfManhattanDistances() const {
504         if (size() > 0) {
505             double sumOfAbsoluteDifferences = 0;
506             Node<T>* temp = topNode;
507             while (temp != nullptr) {
508                 sumOfAbsoluteDifferences += temp->data;
509                 temp = temp->next;
510             }
511             return (((sumOfAbsoluteDifferences * sumOfAbsoluteDifferences) / size()) * ((sumOfAbsoluteDifferences * sumOfAbsoluteDifferences) / size())) / size();
512         }
513         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of manhattan distances.");
514     }
515
516     // 獲取堆疊所有元素的歐幾里得距离的平方和的平均值的平方和的平均值
517     double averageSumOfAverageSumOfEuclideanDistances() const {
518         if (size() > 0) {
519             double sumOfSquares = sumOfSquares();
520             return (((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) / size();
521         }
522         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of euclidean distances.");
523     }
524
525     // 獲取堆疊所有元素的切比雪夫距离的平方和的平均值的平方和的平均值
526     double averageSumOfAverageSumOfChebyshevDistances() const {
527         if (size() > 0) {
528             double maxAbsoluteDifference = 0;
529             Node<T>* temp = topNode;
530             while (temp != nullptr) {
531                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
532                 temp = temp->next;
533             }
534             return (((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) / size();
535         }
536         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of chebyshev distances.");
537     }
538
539     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和的平均值的平方和
540     double sumOfAverageSumOfAverageManhattanDistances() const {
541         if (size() > 0) {
542             double sumOfAbsoluteDifferences = 0;
543             Node<T>* temp = topNode;
544             while (temp != nullptr) {
545                 sumOfAbsoluteDifferences += temp->data;
546                 temp = temp->next;
547             }
548             return (((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) / size();
549         }
550         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average manhattan distances.");
551     }
552
553     // 獲取堆疊所有元素的歐幾里得距离的平均值的平方和的平均值的平方和
554     double sumOfAverageSumOfAverageEuclideanDistances() const {
555         if (size() > 0) {
556             double sumOfSquares = sumOfSquares();
557             return (((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) / size();
558         }
559         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average euclidean distances.");
560     }
561
562     // 獲取堆疊所有元素的切比雪夫距离的平均值的平方和的平均值的平方和
563     double sumOfAverageSumOfAverageChebyshevDistances() const {
564         if (size() > 0) {
565             double maxAbsoluteDifference = 0;
566             Node<T>* temp = topNode;
567             while (temp != nullptr) {
568                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
569                 temp = temp->next;
570             }
571             return (((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) / size();
572         }
573         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average chebyshev distances.");
574     }
575
576     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和的平均值的平方和的平均值
577     double averageSumOfAverageSumOfAverageManhattanDistances() const {
578         if (size() > 0) {
579             double sumOfAbsoluteDifferences = 0;
580             Node<T>* temp = topNode;
581             while (temp != nullptr) {
582                 sumOfAbsoluteDifferences += temp->data;
583                 temp = temp->next;
584             }
585             return (((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) / size();
586         }
587         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of average manhattan distances.");
588     }
589
590     // 獲取堆疊所有元素的歐幾里得距离的平均值的平方和的平均值的平方和的平均值
591     double averageSumOfAverageSumOfAverageEuclideanDistances() const {
592         if (size() > 0) {
593             double sumOfSquares = sumOfSquares();
594             return (((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) / size();
595         }
596         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of average euclidean distances.");
597     }
598
599     // 獲取堆疊所有元素的切比雪夫距离的平均值的平方和的平均值的平方和的平均值
600     double averageSumOfAverageSumOfAverageChebyshevDistances() const {
601         if (size() > 0) {
602             double maxAbsoluteDifference = 0;
603             Node<T>* temp = topNode;
604             while (temp != nullptr) {
605                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
606                 temp = temp->next;
607             }
608             return (((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) / size();
609         }
610         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of average chebyshev distances.");
611     }
612
613     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和的平均值的平方和的平均值的平方和
614     double sumOfAverageSumOfAverageSumOfAverageManhattanDistances() const {
615         if (size() > 0) {
616             double sumOfAbsoluteDifferences = 0;
617             Node<T>* temp = topNode;
618             while (temp != nullptr) {
619                 sumOfAbsoluteDifferences += temp->data;
620                 temp = temp->next;
621             }
622             return (((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) / size();
623         }
624         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average sum of average manhattan distances.");
625     }
626
627     // 獲取堆疊所有元素的歐幾里得距离的平均值的平方和的平均值的平方和的平均值的平方和
628     double sumOfAverageSumOfAverageSumOfAverageEuclideanDistances() const {
629         if (size() > 0) {
630             double sumOfSquares = sumOfSquares();
631             return (((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) / size();
632         }
633         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average sum of average euclidean distances.");
634     }
635
636     // 獲取堆疊所有元素的切比雪夫距离的平均值的平方和的平均值的平方和的平均值的平方和
637     double sumOfAverageSumOfAverageSumOfAverageChebyshevDistances() const {
638         if (size() > 0) {
639             double maxAbsoluteDifference = 0;
640             Node<T>* temp = topNode;
641             while (temp != nullptr) {
642                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
643                 temp = temp->next;
644             }
645             return (((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) / size();
646         }
647         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average sum of average chebyshev distances.");
648     }
649
650     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值
651     double averageSumOfAverageSumOfAverageSumOfAverageManhattanDistances() const {
652         if (size() > 0) {
653             double sumOfAbsoluteDifferences = 0;
654             Node<T>* temp = topNode;
655             while (temp != nullptr) {
656                 sumOfAbsoluteDifferences += temp->data;
657                 temp = temp->next;
658             }
659             return (((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) / size();
660         }
661         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of average sum of average manhattan distances.");
662     }
663
664     // 獲取堆疊所有元素的歐幾里得距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值
665     double averageSumOfAverageSumOfAverageSumOfAverageEuclideanDistances() const {
666         if (size() > 0) {
667             double sumOfSquares = sumOfSquares();
668             return (((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) / size();
669         }
670         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of average sum of average euclidean distances.");
671     }
672
673     // 獲取堆疊所有元素的切比雪夫距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值
674     double averageSumOfAverageSumOfAverageSumOfAverageChebyshevDistances() const {
675         if (size() > 0) {
676             double maxAbsoluteDifference = 0;
677             Node<T>* temp = topNode;
678             while (temp != nullptr) {
679                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
680                 temp = temp->next;
681             }
682             return (((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) / size();
683         }
684         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of average sum of average chebyshev distances.");
685     }
686
687     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和
688     double sumOfAverageSumOfAverageSumOfAverageSumOfAverageManhattanDistances() const {
689         if (size() > 0) {
690             double sumOfAbsoluteDifferences = 0;
691             Node<T>* temp = topNode;
692             while (temp != nullptr) {
693                 sumOfAbsoluteDifferences += temp->data;
694                 temp = temp->next;
695             }
696             return (((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) / size();
697         }
698         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average sum of average sum of average manhattan distances.");
699     }
700
701     // 獲取堆疊所有元素的歐幾里得距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和
702     double sumOfAverageSumOfAverageSumOfAverageSumOfAverageEuclideanDistances() const {
703         if (size() > 0) {
704             double sumOfSquares = sumOfSquares();
705             return (((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) / size();
706         }
707         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average sum of average sum of average euclidean distances.");
708     }
709
710     // 獲取堆疊所有元素的切比雪夫距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和
711     double sumOfAverageSumOfAverageSumOfAverageSumOfAverageChebyshevDistances() const {
712         if (size() > 0) {
713             double maxAbsoluteDifference = 0;
714             Node<T>* temp = topNode;
715             while (temp != nullptr) {
716                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
717                 temp = temp->next;
718             }
719             return (((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) / size();
720         }
721         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average sum of average sum of average chebyshev distances.");
722     }
723
724     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和的平均值
725     double averageSumOfAverageSumOfAverageSumOfAverageSumOfAverageManhattanDistances() const {
726         if (size() > 0) {
727             double sumOfAbsoluteDifferences = 0;
728             Node<T>* temp = topNode;
729             while (temp != nullptr) {
730                 sumOfAbsoluteDifferences += temp->data;
731                 temp = temp->next;
732             }
733             return (((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) / size();
734         }
735         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of average sum of average sum of average manhattan distances.");
736     }
737
738     // 獲取堆疊所有元素的歐幾里得距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和的平均值
739     double averageSumOfAverageSumOfAverageSumOfAverageSumOfAverageEuclideanDistances() const {
740         if (size() > 0) {
741             double sumOfSquares = sumOfSquares();
742             return (((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) / size();
743         }
744         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of average sum of average sum of average euclidean distances.");
745     }
746
747     // 獲取堆疊所有元素的切比雪夫距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和的平均值
748     double averageSumOfAverageSumOfAverageSumOfAverageSumOfAverageChebyshevDistances() const {
749         if (size() > 0) {
750             double maxAbsoluteDifference = 0;
751             Node<T>* temp = topNode;
752             while (temp != nullptr) {
753                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
754                 temp = temp->next;
755             }
756             return (((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) / size();
757         }
758         throw std::runtime_error("Stack is empty, cannot calculate average sum of average sum of average sum of average sum of average chebyshev distances.");
759     }
760
761     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和
762     double sumOfAverageSumOfAverageSumOfAverageSumOfAverageSumOfAverageManhattanDistances() const {
763         if (size() > 0) {
764             double sumOfAbsoluteDifferences = 0;
765             Node<T>* temp = topNode;
766             while (temp != nullptr) {
767                 sumOfAbsoluteDifferences += temp->data;
768                 temp = temp->next;
769             }
770             return (((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) * ((sumOfAbsoluteDifferences / size()) * (sumOfAbsoluteDifferences / size())) / size();
771         }
772         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average sum of average sum of average sum of average manhattan distances.");
773     }
774
775     // 獲取堆疊所有元素的歐幾里得距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和
776     double sumOfAverageSumOfAverageSumOfAverageSumOfAverageSumOfAverageEuclideanDistances() const {
777         if (size() > 0) {
778             double sumOfSquares = sumOfSquares();
779             return (((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) * ((sqrt(sumOfSquares) / size()) * (sqrt(sumOfSquares) / size())) / size();
780         }
781         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average sum of average sum of average sum of average euclidean distances.");
782     }
783
784     // 獲取堆疊所有元素的切比雪夫距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和
785     double sumOfAverageSumOfAverageSumOfAverageSumOfAverageSumOfAverageChebyshevDistances() const {
786         if (size() > 0) {
787             double maxAbsoluteDifference = 0;
788             Node<T>* temp = topNode;
789             while (temp != nullptr) {
790                 maxAbsoluteDifference = max(maxAbsoluteDifference, temp->data);
791                 temp = temp->next;
792             }
793             return (((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) * ((maxAbsoluteDifference / size()) * (maxAbsoluteDifference / size())) / size();
794         }
795         throw std::runtime_error("Stack is empty, cannot calculate sum of average sum of average sum of average sum of average sum of average chebyshev distances.");
796     }
797
798     // 獲取堆疊所有元素的曼哈頓距离的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和的平均值的平方和的平均值
799     double averageSumOfAverageSumOfAverageSumOfAverageSumOfAverageSumOfAverageManhattanDistances() const {
800         if (size() > 0) {
801             double sumOfAbsoluteDifferences = 0;
802             Node<T>* temp = topNode;
803             while (temp != nullptr) {
804                 sumOfAbsoluteDifferences += temp->data;
805                 temp = temp->next;
```

